NLP Assignment-2 Report

Ishaan Singla (24103099)
Tejender (24103014)
Raman Vashisht (24103030)
Manvendra Singh (24103004)

Aim and Overview of the Assignment:

The aim of this assignment is to develop and evaluate a text summarization model using NLP techniques. The project involved training the RNN (or LSTM) model on the CNN/Daily Mail dataset, which contains articles and their corresponding summaries. After preprocessing the data by cleaning and tokenizing the text, I converted it into sequences suitable for model input.

A sequence-to-sequence model with multiple LSTM layers was developed, incorporating an attention mechanism to enhance summary generation. The model was trained using a portion of the CNN/Daily Mail dataset while monitoring performance on a validation set. For final testing, I utilized a separate Wikipedia summary dataset containing 5000 samples to evaluate the model's effectiveness. The evaluation was conducted using ROUGE scores to analyze the quality of the generated summaries.

Part-1: Data Exploration and Model Building:

Task-1: Data Exploration with CNN/Daily Mail Dataset:

1 a) After inspecting the dataset, I identified three columns: 'article', 'highlights', and 'id'. The 'article' column contains the full text, while the 'highlights' column provides the corresponding summaries for each article. This structure facilitated the development of the text summarization model by using articles for training and highlights for validation and evaluation.

1 c) Before exploring the dataset, I performed basic text cleaning to enhance the quality of the analysis. This involved converting the text to lowercase, removing special characters, and tokenizing the text to ensure only words were retained. The cleaning process was applied to both the headlines (summaries) and the articles. By tokenizing and cleaning the text beforehand, I could obtain a clearer understanding of word frequency and sentence lengths, which are crucial for evaluating the model's performance in text summarization. The cleaned tokens were then flattened for subsequent frequency analysis.

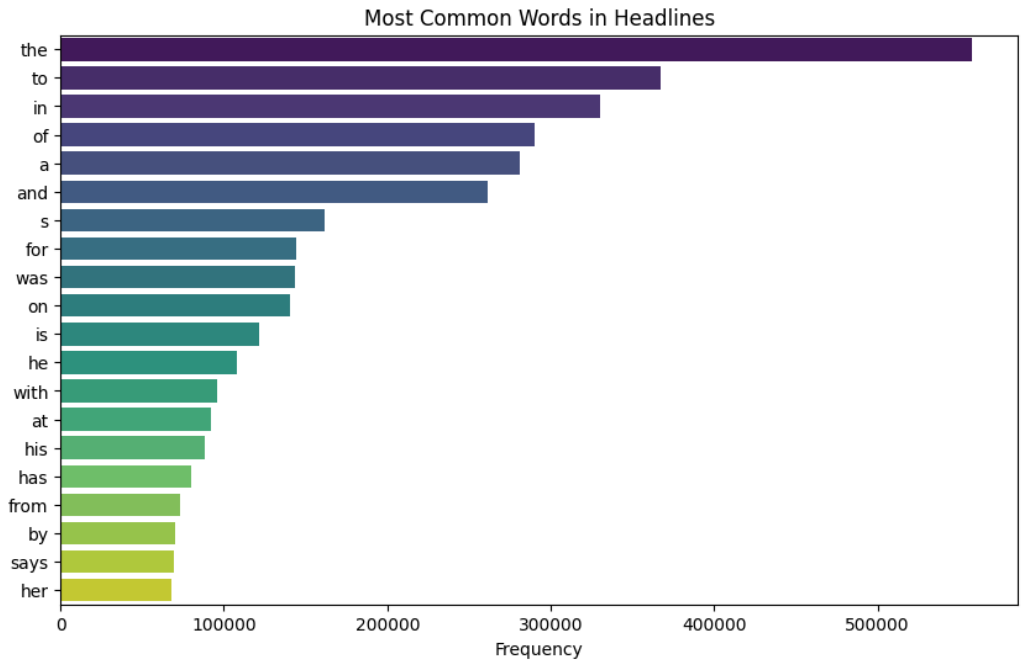
1 b) After exploration I found:

The most frequent word in both articles(text) and highlights(summary) is : the

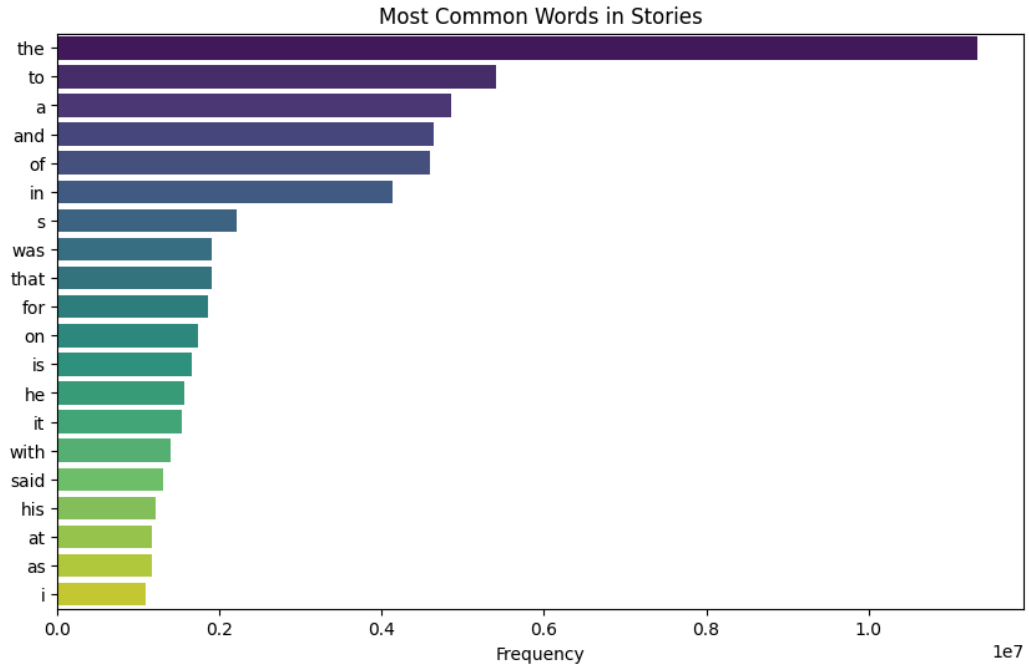
The most frequent article length is : 500-700

The most frequent highlights length is : 10-150

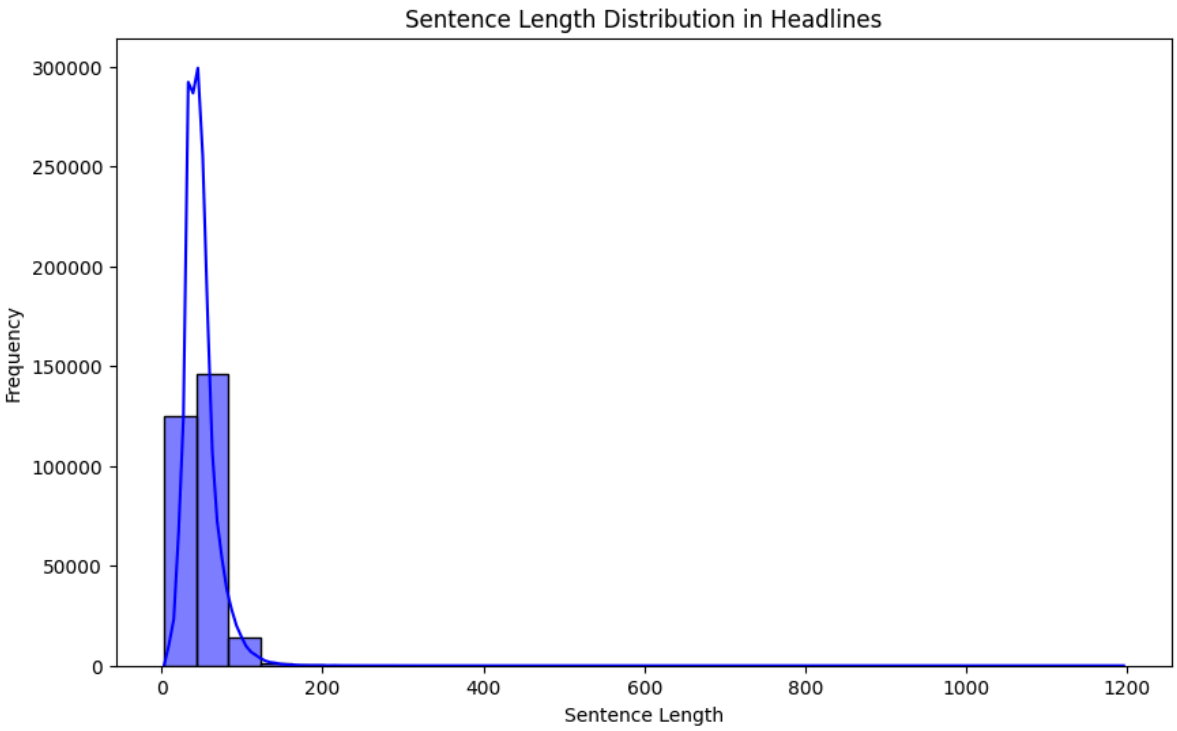
1 d) Visualization of all the findings:

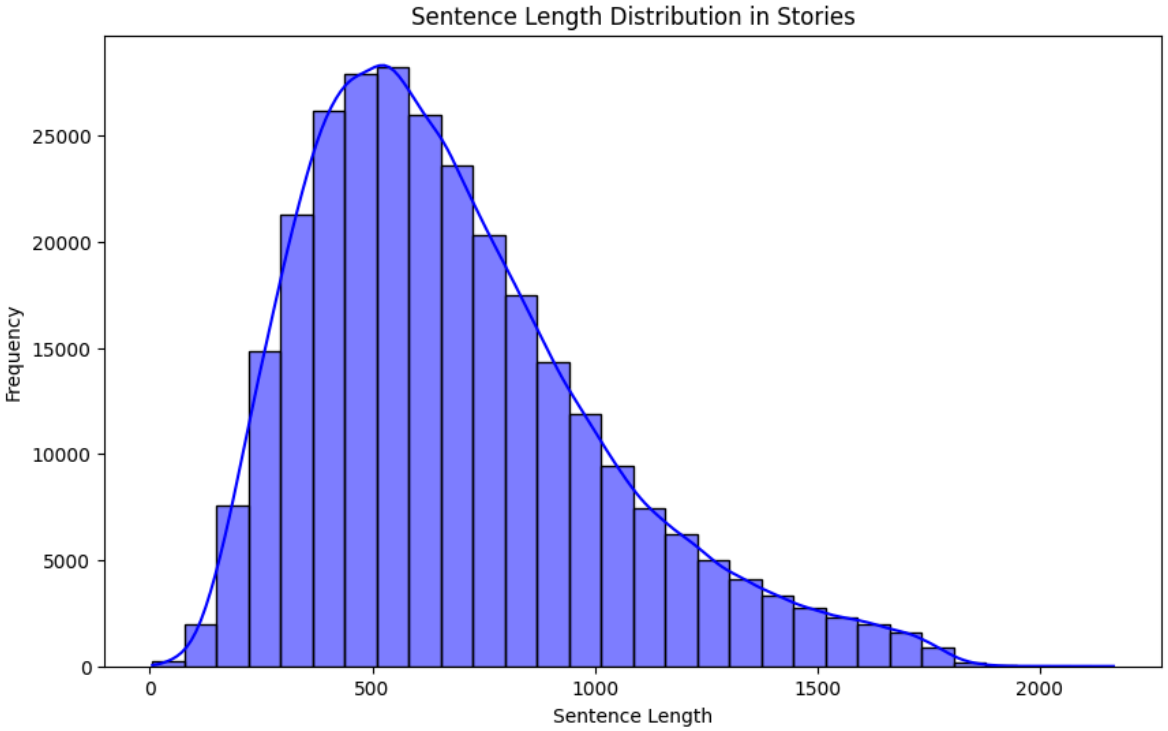


Note: Headlines is basically summary



Note: Stories is basically articles


Sentence Length Distribution in Headlines
Frequency
Sentence Length
0
50000
100000
150000
200000
250000
300000
0
200
400
600
800
1000
1200


Sentence Length Distribution in Stories
Frequency
Sentence Length
0
5000
10000
15000
20000
25000
0
500
1000
1500
2000

Task 2: Build and Train Seq2Seq Model for Text Summarization:

To build the Seq2Seq model for text summarization, I experimented with two approaches. Initially, I developed a standard Seq2Seq model using LSTM layers without incorporating an attention mechanism. However, the evaluation scores were significantly lower than expected. Based on feedback received via email, I decided to enhance the model by integrating an attention layer mechanism along with pre-trained word embeddings. This adjustment aimed to improve the model's ability to focus on relevant parts of the input sequence during decoding, thereby enhancing the summarization performance, however unfortunately the results didn't improve significantly but here is the detailed explanation of what I did in the 2 approaches.

Approach-1 : Without using Attention Layer Mechanism

a) Preprocessing:-

1. Dataset Loading:

- The full CNN/Daily Mail dataset is loaded using the Hugging Face `datasets` library.
- The dataset is reduced to a 10% sample, ensuring that a representative subset is selected for training while reducing the computational load.

2. Data Cleaning:

- A DataFrame is created from the dataset, containing 'text' (articles) and 'summary' (highlights) columns.
- The dataset is filtered to remove duplicate entries and rows with missing values.
- A mapping dictionary is used to expand English contractions (e.g., "he's" to "he is").
- HTML tags, possessive forms, and non-alphabetic characters are removed from the text.
- Optional stopword removal and token filtering are applied to clean the data.

3. Length Analysis:

- The number of words in each text and summary is calculated to determine suitable maximum lengths.
- Articles longer than 700 words and summaries longer than 100 words are excluded, setting these as the maximum lengths.

4. Tagging for Sequence Decoding:

- "START" (`sostok`) and "END" (`eostok`) tokens are added to summaries to aid in the decoding process during training.

5. Data Splitting:

- The dataset is split into training and testing sets (80% training, 20% testing) with shuffling for randomization.

6. Tokenization and Vocabulary Building:

- Tokenizers for both articles and summaries are created using the `Tokenizer` class from Keras.

- A threshold is set for considering rare words (those appearing fewer than a specified number of times), with the code calculating the percentage of rare words and their coverage in the dataset.
- The vocabulary size is adjusted based on the frequency of words.

7. Sequence Conversion and Padding:
- The text and summary sequences are converted to integer sequences based on the tokenizers.
- The sequences are padded to the maximum lengths defined earlier, ensuring uniform input sizes.

8. Vocabulary Size Calculation:
- The final vocabulary size for both articles and summaries is determined.

b) Building the seq2seq Architecture:-

• Encoder: A unidirectional LSTM with an embedding layer was used to capture the meaning of the input text. The encoder has a total of **5 LSTM layers**.
• Decoder: A unidirectional LSTM decoder was employed to help the model focus on relevant sections of the input when generating the summary. The decoder part has only **1 layer of LSTM**.
• TimeDistributed Layer: The overall model architecture also has a TimeDistributed layer to provide the final output as a sequence of tokens.

```
Model: "functional_1"
```

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 700)	0	-
embedding (Embedding)	(None, 700, 256)	14,496,256	input_layer[0][0]
lstm (LSTM)	[(None, 700, 256), (None, 256), (None, 256)]	525,312	embedding[0][0]
lstm_1 (LSTM)	[(None, 700, 256), (None, 256), (None, 256)]	525,312	lstm[0][0]
lstm_2 (LSTM)	[(None, 700, 256), (None, 256), (None, 256)]	525,312	lstm_1[0][0]
input_layer_1 (InputLayer)	(None, None)	0	-
lstm_3 (LSTM)	[(None, 700, 256), (None, 256), (None, 256)]	525,312	lstm_2[0][0]
embedding_1 (Embedding)	(None, None, 256)	3,159,552	input_layer_1[0]…
lstm_4 (LSTM)	[(None, 700, 256), (None, 256), (None, 256)]	525,312	lstm_3[0][0]
lstm_5 (LSTM)	[(None, None, 256), (None, 256), (None, 256)]	525,312	embedding_1[0][0… lstm_4[0][1], lstm_4[0][2]
time_distributed (TimeDistributed)	(None, None, 12342)	3,171,894	lstm_5[0][0]

```
Total params: 23,979,574 (91.47 MB)
```

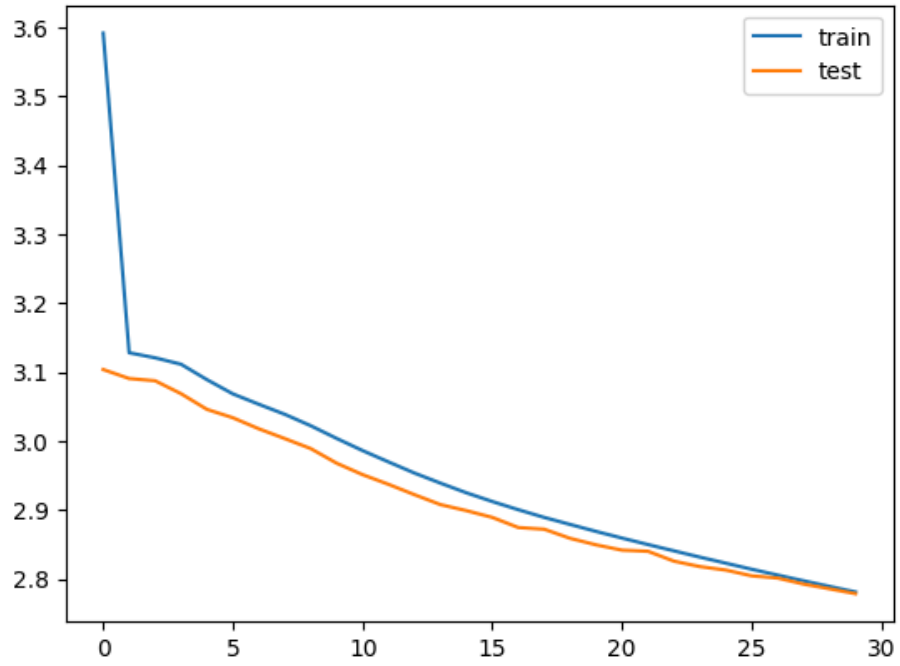
```
Trainable params: 23,979,574 (91.47 MB)
```

```
Non-trainable params: 0 (0.00 B)
```

• Hyperparameters:
    o Embedding size: 256
    o Hidden units (LSTM): 256
    o Batch size: 128
    o Optimizer: rmsprop
    o Loss: sparse_categorical_crossentropy

c) Training and Evaluation:-

• Training Process: The model was trained on 30 epochs with early stopping based on the validation loss. Also, the training data was split 80%-20% into training and validation sets.



• Evaluation Metric: ROUGE-2 and ROUGE-L scores were used to measure the quality of generated summaries.
• Results on Test Data (CNN/Daily Mail):
    o ROUGE-2 Score: 0.0202
    o ROUGE-L Score: 0.1113

Approach-2 Using Attention Layer and Pre-trained Embeddings

a) Preprocessing:-

In approach 2, the preprocessing remains largely similar to approach 1, with the main difference being the use of the pre-trained Google News word2vec model. The model was loaded using the `gensim` library from the `GoogleNews-vectors-negative300.bin` file, providing 300-dimensional word embeddings to better capture semantic relationships in the text.

Imp Note: For running the file **nlp-assgn2-part1-task2** please download the .bin file and add it as an input, the link for downloading the file and how to add is mentioned at the end of the report in detail.

b) Building the seq2seq architecture:-

• Encoder: A unidirectional LSTM with an embedding layer was used to capture the meaning of the input text. The encoder has a total of **3 LSTM layers**.

• Decoder: A unidirectional LSTM decoder was employed to help the model focus on relevant sections of the input when generating the summary. The decoder part has only **1 layer of LSTM**.

• Attention Layer: The addition of the attention mechanism (Bahdanau attention) allows the model to dynamically focus on different parts of the input sequence during each decoding step, enhancing the model's ability to generate more contextually relevant summaries.

• TimeDistributed Layer: The overall model architecture also has a TimeDistributed layer to provide the final output as a sequence of tokens.

```
Model: "functional_1"
```

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 1000)	0	-
embedding (Embedding)	(None, 1000, 300)	38,653,800	input_layer[0][0]
lstm (LSTM)	[(None, 1000, 256), (None, 256), (None, 256)]	570,368	embedding[0][0]
input_layer_1 (InputLayer)	(None, None)	0	-
lstm_1 (LSTM)	[(None, 1000, 256), (None, 256), (None, 256)]	525,312	lstm[0][0]
embedding_1 (Embedding)	(None, None, 300)	14,100,000	input_layer_1[0]…
lstm_2 (LSTM)	[(None, 1000, 256), (None, 256), (None, 256)]	525,312	lstm_1[0][0]
lstm_3 (LSTM)	[(None, None, 256), (None, 256), (None, 256)]	570,368	embedding_1[0][0… lstm_2[0][1], lstm_2[0][2]
attention_layer (AttentionLayer)	[(None, None, 256), (None, None, 1000)]	131,328	lstm_2[0][0], lstm_3[0][0]
concat_layer (Concatenate)	(None, None, 512)	0	lstm_3[0][0], attention_layer[…
time_distributed (TimeDistributed)	(None, None, 12851)	6,592,563	concat_layer[0][…

```
Total params: 61,669,051 (235.25 MB)
```

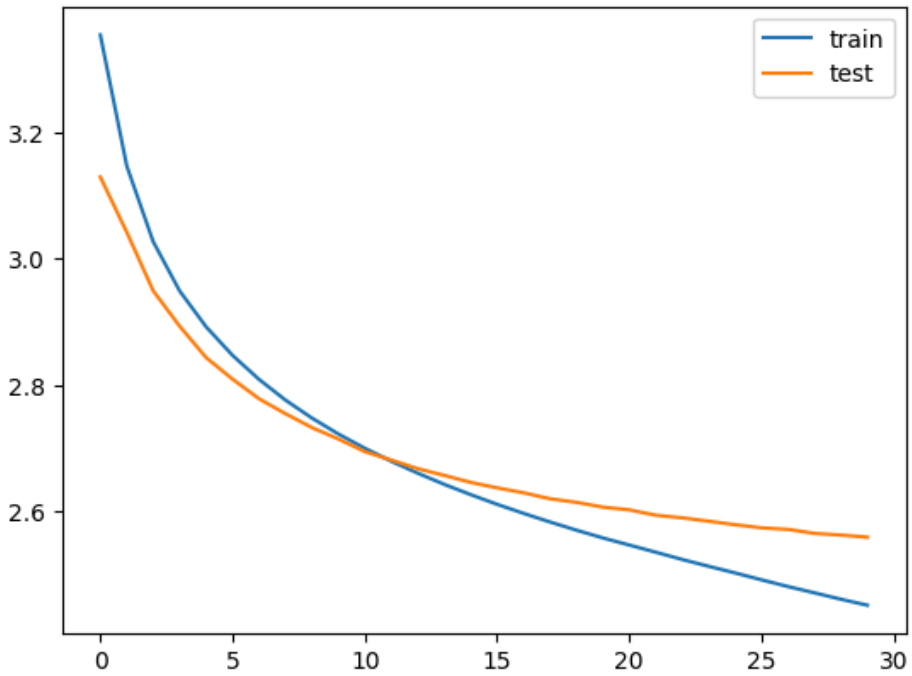
```
Trainable params: 8,783,923 (33.51 MB)
```

```
Non-trainable params: 52,885,128 (201.74 MB)
```

• Hyperparameters:
    - o Embedding size: 300
    - o Hidden units (LSTM): 256
    - o Batch size: 64
    - o Optimizer: rmsprop
    - o Loss: sparse_categorical_crossentropy

c) Training and Evaluation:-

• Training Process: The model was trained on 30 epochs with early stopping based on the validation loss. Also, the training data was split 80%-20% into training and validation sets.



• Evaluation Metric: ROUGE-2 and ROUGE-L scores were used to measure the quality of generated summaries.
• Results on Test Data (CNN/Daily Mail):
    - o ROUGE-2 Score: 0.0116
    - o ROUGE-L Score: 0.1155

Final Observation

The experiments conducted in Task 2 revealed significant insights into the model's performance on text summarization. In Approach 1, which did not incorporate an attention layer, the model achieved ROUGE scores of ROUGE-2 (0.0202) and ROUGE-L (0.1113). While these scores indicate a basic level of summarization capability, the primary limitation of this approach was the generation of identical and irrelevant summaries for each input test sentence. This outcome

highlighted a crucial issue: the model struggled to generate meaningful and diverse summaries, which significantly impacted the quality of the output.

To address this problem, enhancements were made in Approach 2 by incorporating word2vec embeddings and an attention layer. The attention mechanism was added to improve the model's ability to focus on relevant sections of the input during summary generation, potentially leading to more accurate and contextually appropriate summaries. Despite these changes, the ROUGE scores did not show significant improvement. Although there was a slight increase in ROUGE-L from 0.1113 to 0.1155, ROUGE-2 decreased from 0.0202 to 0.0116. While these metrics did not indicate substantial progress, it is important to note that ROUGE scores are not always the most reliable indicators of summarization quality, especially in terms of relevance and diversity.

The more pressing issue, however, was that the problem of generating the same output for each test sentence persisted. After careful analysis, it was observed that even after training on 10% of the CNN/Daily Mail dataset for 30 epochs, the model's loss function did not converge adequately to produce varied and meaningful summaries. The training process was constrained by resource limitations, with each epoch taking approximately 500-1000 seconds to complete. Given the large number of trainable parameters and the substantial size of the dataset, increasing the number of epochs was not a feasible solution.

To further evaluate the model's capabilities, a smaller dataset was used to alleviate resource constraints. With this new dataset, the model's behavior improved slightly, as it started generating different outputs for each test sentence, with some outputs being more relevant to the input context. Although there was a decrease in the ROUGE scores when trained on the smaller dataset, the variation in the output indicated that the model was at least making some progress in learning meaningful patterns.

To validate the model's generalization ability, experiments were conducted using the Wikipedia summary as input, after training on the new, smaller dataset. The details of the new dataset, the updated ROUGE scores, and the results from testing with the Wikipedia summaries are further elaborated in the explanation of Part 2 below.

In conclusion, the experiments highlighted the challenges in training complex models for text summarization with limited computational resources. The observations suggest that while enhancements like word2vec embeddings and attention layers can contribute to improvements, addressing issues such as model overfitting, training time, and dataset size requires careful tuning and potentially different approaches to achieve optimal results.